Hard Margin Support Vector Machines

Aaron Avram

May 23 2025

Introduction

I will go through some of the derivations of my SVM implementation. This will not include any details on how I implemented my NLP for the model as since the model itself is quite rudimentary so was the NLP I employed. Otherwise I will go through the support vector machine construction then the optimization strategy.

Construction

The basic idea behind an SVM is to divide the input training data, given as a list of n vectors, into two categories by constructing a hyperplane in the input space such that input vectors on one side are in one category and those on the other side are in the other category denoted by the labels $\{-1, 1\}$. Formulating this more precisely: Given a hyperplane parametrized by the vector x: $w^T x + b = 0$ for some vector w and real number b, given an input vector x_i, with label y_i if $y_i = -1$ we want $w^T x_i + b \leq 0$ and likewise if $y_i = 1$ we would want $w^T x_i + b \geq 0$. It is reasonable to also include a threshold a, which for convenience we can choose to be 1, so there is a clearer separation between categories. Now we have a width where there are no points. In other words the space in between the hyperplanes $w^T x_i + b = 1$ and $w^T x_i + b = -1$ is empty. Now the optimization strategy is clear: maximize this width l to achieve the best possible separation between the two classes. Now we must find a formula for l in terms of w and b

First notice that w is perpendicular to our hyperplane. This can by picking two vectors x_0, x_1 whose tip lies on the hyperplane. Since they lie on the hyperplane, x_0, x_1 satisfy the above relation that parametrizes it, so $w^T x_0 = w^T x_1 = -b$. Thus $w^T(x_1 - x_0) = 0$. Since $x_1 - x_0$ is parallel to the hyperplane, w is perpendicular to it.

Now we can formulate the margin width in terms of w, where the margin width is equal to the norm of the difference of the multiple of w that intersects the lower boundary $w^T x_i + b = -1$ and the multiple of w that intersects the upper boundary $w^T x_i + b = 1$. Which occurs for scalars $\frac{-1-b}{||w||^2}$ and $\frac{1-b}{||w||^2}$. Thus norm of the difference is:

$$l = ||\frac{-1-b}{||w||^2}w - \frac{-1-b}{||w||^2}w|| = ||\frac{2w}{||w||^2}|| = \frac{2}{||w||}$$

Thus we must maximize $\frac{2}{||w||}$ which is equivalent to minimizing $||w||$ or minimizing $\frac{1}{2}||w||^2$ which we will see soon is more convenient. However, we still want our predictor to separate the input properly thus we still have the constraint that $w^T x_i + b \geq 1$ for $y_i = 1$ and $w^T x_i + b \leq -1$ for $y_i = -1$. Which can be combined into a single constraint: $y_i(w^T x_i + b) \geq 1$.

Optimization

Now we have a clear optimization goal and we can proceed using the method of Lagrange Multipliers, which says that minimizing our function $J(w) = \frac{1}{2}||w||^2$ with the constraints
$g_i(w, b) = 1 - y_i(w^T x_i + b) \leq 1$ for $1 \leq i \leq n$, which we can package as a vector g, is equivalent to the following:

$$\max_{\lambda} \min_{w} \mathcal{L}(w, b, \lambda)$$
$$\mathcal{L}(w, b, \lambda) := \frac{1}{2}||w||^2 + \lambda^T g$$

For later use we can compute the gradient of the Lagrangian with respect to b and w:

$$\begin{aligned}
\nabla_w(\mathcal{L}(w, b, \lambda)) &= \nabla_w(\frac{1}{2}||w||^2 + \lambda^T g) \\
&= \nabla_w(\frac{1}{2}||w||^2 + \sum_{i=1}^{n} \lambda_i(1 - y_i(w^T x_i + b))) \\
&= w - \sum_{i=1}^{n} \lambda_i y_i x_i
\end{aligned}$$

$$\begin{aligned}
\nabla_b(\mathcal{L}(w, b, \lambda)) &= \nabla_b(\frac{1}{2}||w||^2 + \lambda^T g) \\
&= \nabla_b(\frac{1}{2}||w||^2 + \sum_{i=1}^{n} \lambda_i(1 - y_i(w^T x_i + b))) \\
&= -\sum_{i=1}^{n} \lambda_i y_i
\end{aligned}$$

Since the first gradient is zero at a minimum of the Lagrangian with respect to w, we can substitute: $w = \sum_{i=1}^{n} \lambda_i y_i x_i$, which solves the minimization problem.

$$\begin{aligned}
\mathcal{D}(\lambda) &= \min_{w} \mathcal{L}(w, b, \lambda) \\
&= \frac{1}{2}||w||^2 + \sum_{i=1}^{n} \lambda_i(1 - y_i(w^T x_i + b)) \\
&= \frac{1}{2}(\sum_{i=1}^{n} \lambda_i y_i x_i)^T(\sum_{i=1}^{n} \lambda_i y_i x_i) - \sum_{i=1}^{n} y_i((\sum_{j=1}^{n} \lambda_j y_i x_j)^T x_i) + \sum_{i=1}^{n} \lambda_i + b\sum_{i=1}^{n} y_i
\end{aligned}$$

If we let $H := yy^T \bigodot XX^T$ where X is the matrix formed such that $X_i = x_i^T$ then:

We can omit the term not containing a λ

$$\mathcal{D}(\lambda) = \frac{1}{2}\lambda^T H\lambda - \lambda^T H\lambda + \lambda^T 1$$
$$-\mathcal{D}(\lambda) = \frac{1}{2}\lambda^T H\lambda - \lambda^T 1$$

Based on the Karush-Kuhn-Tucker conditions, we have to solve this equation such that $\sum_{i=1}^{n} \lambda_i y_i = 0$ as that is required for a minimum of the lagrangian with respect to b and $\lambda \geq 0$. Once we find the optimal λ, w can be recovered using the above gradient identity: $w = \sum_{i=1}^{n} \lambda_i y_i x_i$. To solve this, we have to maximize D (or minimize $-D$) with respect to λ with the given constraints. To do this I performed projective gradient descent where I computed the gradient:

$$\nabla_\lambda(-\mathcal{D}(\lambda)) = H\lambda - 1$$

and at every step of gradient descent I projected the new λ value on to the orthogonal complement of $\sum_{i=1}^{n} \lambda_i y_i = 0$ and applied the ReLU to λ elementwise. This worked moderately well, leaving me with a pretty good λ value and a moderately low gradient. To recover the value of b, I found all of the support vectors of the model (The vectors that lie on the decision boundaries) and computed $y_i - x_i^T w_i$ for each of them and averaged the result. With this, I was able to make my predictions!